A Multi-Participant Co-located WebXR System

KERU WANG, New York University, USA
PINCUN LIU, New York University, USA
BRAYTON LORDIANTO, New York University, USA
YUSHEN HU, New York University, USA
ZHU WANG, New York University, USA
KEN PERLIN, New York University, USA

This paper presents the implementation and challenges faced in developing a co-located multi-participant WebXR system that enables users in the same physical space to share a common video passthrough extended reality experience. The system is designed to handle multiple physical rooms, each identified by a unique room ID, allowing for distinct virtual content and interactions within each room. We categorize the challenges encountered into high-level tasks and ways of interaction that typically occur in a co-located XR setting. The system additionally provides a framework for developers to easily create their own XR experiences with WebXR. The capability to create collaborative experiences is facilitated through a custom-designed Node.js server and employing a shared blackboard model. This model comprises a set of shared state variables, ensuring consistency across all connected clients. The synchronization mechanism operates by relaying messages from each client to the server that communicate changes in the values of shared state objects, which the server then relays to all other clients. The system architecture is outlined, highlighting the mechanisms employed to meet the needs of multiple simultaneous participants. The paper contributes insights into implementing a scaleable and adaptable co-located WebXR system, powered by a custom-made WebXR renderer, offering a foundation for creating engaging and useful shared interactive experiences. The described architecture provides a framework for future developments in scaleable collaborative co-located XR systems.

CCS Concepts: • **Computer systems organization** → **Embedded systems**; *Redundancy*; Robotics; • **Networks** → Network reliability.

Additional Key Words and Phrases: Extended Reality, Collocated, Multiplayer

ACM Reference Format:
Keru Wang, Pincun Liu, Brayton Lordianto, Yushen Hu, Zhu Wang, and Ken Perlin. 2018. A Multi-Participant Co-located WebXR System. In . ACM, New York, NY, USA, 9 pages. https://doi.org/XXXXXXX.XXXXXXX

1 INTRODUCTION

Immersive technologies have been undergoing a paradigm shift, with Extended Reality (XR) emerging as an exciting alternative to Virtual Reality(VR) and Augmented Reality(AR), particularly in terms of multi-person immersion and interaction[9]. Unlike VR platforms, which focus on complete digital immersion, or Augmented Reality (AR), which overlays digital images onto the user's view of the real world [21, 26, 27], XR seamlessly integrates virtual content with the real environment and allows for dynamic interactions with physical spaces [4, 7, 16, 21].

Traditional XR experiences often focus on individual immersion. As XR technologies find applications across diverse fields such as education, training, gaming, and collaborative workspaces, the demand for multi-person experiences has grown. This paper addresses the critical need for a robust solution to enable users sharing the same physical environment to partake in a seamless and unified XR experience. The significance lies in transcending the boundaries of traditional single-user XR interactions and unlocking the potential for collaborative engagement within co-located settings. The implementation of a co-located multi-participant WebXR system [5] is an important step in filling this need, offering a transformative approach that not only addresses the challenges encountered but also presents a scalable and adaptable framework.

The motivation behind the development of multi-person co-located XR systems comes from the inherent human desire for shared experiences [15, 20, 28]. While XR initially gained attention for its ability to immerse individuals in digital worlds, the focus now has been shifting to connecting users in a shared virtual space. Collaborative XR experiences open up possibilities for social interaction, teamwork, and shared exploration, going beyond the confines of traditional single-user interactions [2, 29, 32]. XR-based co-located experiences leverage the physical proximity of participants, allowing for spatial awareness and physical interactions within the shared virtual environment. Users can directly see and interact with each other instead of avatars, enhancing the sense of presence and social connection. In addition, it allows for the observation and analysis of group dynamics. In teamwork settings such as education and training, instructors can observe how participants collaborate and learn from each other, providing valuable insights for instructional improvement [23].

Game engines like Unity and Unreal Engine are popular for XR development due to their comprehensive tools and cross-platform support, but they require application downloads and developer compilations, limiting accessibility. In contrast, WebXR delivers XR experiences through web browsers, enhancing cross-platform capability and eliminating the need for installation or compilation. This approach streamlines development, supports live coding, and simplifies content distribution via URLs. Users can access XR content instantly on any device with a web browser, and developers can make real-time updates without user intervention.

The system that will be described in this paper was developed for this need for research and experimentation with co-located multi-person XR experiences. Specifically, the system provides a platform

Conference acronym 'XX, June 03–05, 2018, Woodstock, NY

ACM ISBN 978-1-4503-XXXX-X/18/06…$15.00
https://doi.org/XXXXXXX.XXXXXXX

for programmers and developers to easily create XR experiences with minimal effort to synchronize experiences between users, and for users to seamlessly enter these immersive, mixed-reality experiences. This paper will outline the implementation details of our system. Then, the paper will describe various experiences that were created on the platform to showcase the rendering capabilities and use cases of our system. In general, this paper offers the following contributions:

(1) presenting the design and implementation of a co-located multi-participant WebXR system, showcasing a practical application of extended reality technologies in shared physical spaces.
(2) presenting a solution for a customizable node.js server. The server is facilitated through a shared blackboard model which features shared state variables and ensures consistency across all connected clients. The server's customizable nature and versatility make it easily adaptable for various applications and allow developers to tailor it to specific needs.
(3) exploring challenges for co-located multi-participant setup and presenting solutions for the challenges including space synchronization between clients.
(4) exploring applications and scenarios that the system can be used for.

2 RELATED WORK

XR covers a wide range of technologies, including virtual reality (VR) and augmented reality (AR). VR fully immerses users in a virtual world and makes them feel like stepping into a different world, while AR is about enhancing the world by having digital information displayed in your surroundings [4, 6, 7, 14, 23].

In XR, collaborative applications allow users to learn or work with others when they are co-located or even if they are far away physically. XR makes this possible by providing a shared space where everyone can see and interact with the same things, just like playing a multiplayer game. This way, people can communicate, share ideas, and solve problems together, creating a more engaging and connected experience [2, 3, 25]. Co-located multi-person XR experiences can enhance spatial awareness and social interactions by creating shared virtual environments, where users can engage in real-time discussions, maintain the spatial state of the surroundings, and contribute to the collective experience through interactive elements [17, 22, 24].

Space synchronization plays a significant role in shared environments. In order to create a seamless and immersive XR landscape for everyone involved, previous works have explored the methods involving spatial anchors, such as QR code [6], Simultaneous Localization and Mapping (SLAM) [10], etc. Xu et al. introduced a calibration method for different users' headsets to synchronize the space by registering the same user's right hand as a calibration anchor [30]. However, our system's space synchronization doesn't depend on SLAM tracking, eliminating the limitation imposed by privacy concerns on most headset platforms. Our system simplifies the synchronization process by enabling users to register an arbitrary calibration anchor through a straightforward action. Each user simultaneously squeezes both controllers at a known room location, establishing a coordinate system with the origin at the midpoint between the controllers. This registration action, performed without camera access, ensures a permanent synchronization of the user's frame of reference with the physical room based on the recorded boundary information, providing a secure and privacy-friendly solution for space synchronization in shared XR environments.

3 SYSTEM DESIGN

In designing our co-located XR system, we began by thoroughly analyzing and summarizing the specific user needs inherent to such an experience. This comprehensive understanding enabled us to methodically develop feature requirements that are directly tailored to meet these needs, ensuring a system that is both technically robust and user-centric.

3.1 Analysis of User Needs in Co-located XR Systems

We have summarized four major user needs in a co-located XR system, listed as **N1**, **N2**, **N3**, and **N4** in the following sections:

N1: Seamless Physical-Virtual Integration. XR environments inherently blend the real and virtual worlds. Users expect a cohesive experience in which the connection between these worlds is fluid and natural. The integration between the real and the virtual enhances the sense of presence and immersion, which is vital to the effectiveness of XR experiences. This becomes particularly crucial in a co-located setting in which multiple users share the same physical space, to ensure a seamless collaborative experience [2, 6, 10, 14, 23, 25, 30].

N2: Stable and Consistent Performance. Consistency and reliability are fundamental to user trust and satisfaction in any technological system. In multi-participant XR, where users are often deeply immersed in a shared experience, unexpected performance issues can be particularly jarring and disruptive. Stability is also crucial in a research context, where reproducibility and reliability of results are key. Co-located scenes demand consistent performance across different sessions and setups ensuring that all participants have a uniformly high-quality experience [2, 6, 26, 27].

N3: Real-Time Interaction and Low Latency. The immersive nature of XR demands real-time rendering and interaction to maintain the feeling of the presence of perceived virtual objects. Any latency can disrupt this feeling, causing a break in immersion and potentially leading to disorientation or discomfort. In a multi-person environment, low latency is essential for synchronous collaboration and interaction, as delays can hinder effective communication and teamwork [2, 6, 10].

N4: Rendering. Our system is, at the highest level, an XR developer environment. A key objective for an effective XR developer platform is to facilitate rendering that is both realistic and true to the vision of the artist or creator of the experience. To achieve this, our system must be capable of high-quality rendering and handling a variety of object types. This is essential for creating an immersive experience for users, and the importance is magnified in a multi-person, co-located XR environment, where the fidelity and quality of computer graphics can significantly influence the overall experience of all participants [19].

3.2 Feature Design

To address the identified user needs, our feature design incorporates specific functionalities that directly cater to the seamless integration of physical and virtual spaces, performance stability, real-time interaction, and accessibility.

Spatial Mapping and Calibration (N1). To achieve seamless physical-virtual integration, we implement advanced spatial mapping and calibration techniques. This feature allows the XR system to accurately map the physical environment and align it with the virtual world, ensuring a cohesive experience for multiple users sharing the same space.

Robust Server Architecture and Optimization (N2, N3). The backbone of stable and consistent performance, as well as real-time interaction with low latency, is a robust server architecture. This includes optimization for high concurrency and efficient data handling, ensuring that performance remains consistent and responsive, even with multiple users and complex virtual environments.

Adaptive Network Handling and Data Compression (N2, N3). Given the importance of stable performance and low latency, our system includes adaptive network handling and data compression techniques. These features ensure that even in environments with varying network conditions, the system maintains a high level of performance, reducing the likelihood of latency or disconnection.

Real-Time Synchronization Protocol (N3). A dedicated protocol for real-time synchronization is implemented to ensure that all users in the co-located space experience the virtual environment in sync. This is crucial for collaborative tasks and interactive experiences, where timing and coordination are key.

Dual Rendering Pipelines for Diverse Quality Needs (N2, N3, N4). To cater to the need for rendering content not only in real-time but also with high resolution, our system integrates two distinct rendering pipelines. One is optimized for simple and fast geometry rendering, ideal for general use and quick interactions. The other focuses on high-resolution rendering, perfect for detailed imagery and text. This dual approach allows the system to dynamically adjust rendering quality based on the user's immediate needs and preferences. More details on this feature will be elaborated in a later section.

Each feature of our co-located XR system is designed to address the specific user needs identified, ensuring an immersive, reliable, and accessible virtual experience for all participants.

4 IMPLEMENTATION

4.1 Modeler and renderer

We created our own custom modeler and renderer based on WebXR, which we export as a module to be used by all of our content. The renderer provides an API that controls and exposes the data of Model objects in the scene for contributors and developers to easily create XR experiences. A self-made renderer and rendering API was chosen over existing frameworks such as A-Frame or Three.js because it allowed us the flexibility to implement custom modeling/rendering features that are particularly useful for co-located XR.

4.2 Clipping away the virtual world

There are occasions when we wish a real-world object, such as the user's hand, to be displayed in front of all rendered computer graphics. Yet the Meta Quest 3 [11] does not provide any support for this in WebXR. But if the position and shape of the real-world object are known, we can achieve the same effect by suppressing CGI rendering at the location of that object.

For example, since the Quest 3 provides real-time joint data for the hand, we can use that data to create a hand-shaped pixel mask and choose to not render any computer graphics for those pixels.

Given any pixel for which we wish the participant to see only video passthrough, we can use every CGI object's position "vPos" in the fragment shader relative to the participant's eye (a righthand coordinate system in which the eye is located at the origin, and the forward direction perpendicular to the headset is the negative z-axis). We can then compute in the fragment shader:

```
vec2 uv = vPos.xy / (vPos.z + 0.2);
```

where the constant 0.2 is determined by the value of the projection matrix in the underlying WebXR implementation. For all objects in the scene, we can then apply the same clipping filter:

```
if (clipAway(uv))
   discard;
```

4.3 Texture-mapped 3D objects and Layers

The utilization of texture-mapped 3D objects in WebXR offers a significant advantage in rendering a multitude of objects around a scene, enhancing the immersive experience. This approach is particularly effective for creating a realistic and interactive 3D environment. However, the issue arises when users require detailed information, such as reading text or examining images closely. The inherent limitations of WebXR-rendered textures in terms of quality and computational demands become a barrier to conveying detailed information effectively.

To address this challenge, the integration of the WebXR Layers feature presents a powerful solution. When a user approaches an object for a closer look, transitioning to Layers allows for the display of high-quality texts and images. This switch ensures that the detailed information is presented in a clear, legible, and visually appealing manner. By leveraging Layers in this context, users can interact with texture-mapped 3D objects in the scene and then seamlessly transition to a detailed view using Layers for specific content, like reading a page in a virtual book or examining high-resolution images.

This combination of texture-mapped 3D objects for general scene rendering and Layers for detailed information viewing not only preserves the immersive quality of the 3D environment but also significantly enhances the user experience by providing high-quality, detailed content exactly when and where it's needed.

4.4 Building 3D from 2D Web graphics

One benefit of using WebXR is that we have access to the native graphical libraries of the Web. For example, we can access the canvas 2D context and use that as a texture source. We have built upon this capability by allowing content to be authored by drawing 2D objects such as text lines and polygon fills, but extending the user

interface so that for each of these operations, the author can add a z component.

Behind the scenes, we map these 3D drawing operations to their 2D native equivalents, project them onto a 2D canvas from the perspective of each eye of the participant, and sort all drawing operations at each rendered frame so that they are performed in back-to-front sorted order. This gives an author the ability to quickly and easily create animated 3D technical diagrams that contain text labels, arrows, and other useful components, but that appear to all participants to be 3D objects in the room, as can be seen in the accompanying video.

4.5 Live coding

One of the advantages of using WebXR is that it allows all participants to perform live programming on the shared world in Javascript from inside the XR experience. When any participant makes a change to the underlying code that describes the scene, this change can be made immediately visible in the 3D scene shared by all participants.

4.6 Use of transparency

Because we cannot have people in front of CGI, we were concerned that we could not implement a giant chess game. But the solution turned out to be simple: We made the chess pieces transparent. We found that all users simply accept that the world of the chess game is transparent, and they do not perceive that anything is amiss. This is an important principle, because it will generalize to other shared co-located room-scale experiences.

4.7 Representing the user

Because the real world always appears behind the CGI, we needed a way to focus the user's attention on a CGI object that acts as a proxy for the controller. We came upon an effective solution in the form of a virtual ping-pong ball that floats in the air just beyond the end of the controller. When the user presses the controller trigger, the virtual ball lights up from within.

Whenever a user puts down their controller and instead uses their hands, we superimpose transparent CGI fingers over the user's actual fingers, using data from the native hand-tracking API on the Meta Quest 3. This gives the user a representation of their own fingers to focus on, which appears correctly in front of the CGI objects they are manipulating, even though the video passthrough of the user's own hands is obscured by those CGI objects.

4.8 Supporting shared objects (Model/View)

In order to reduce communication bandwidth, we use a Model/View architecture. The data objects shared between clients are not the renderable WebXR objects themselves, but rather a descriptive Model of the scene — the minimal abstract representation needed for each Web client to reconstruct their own View of the scene. For example, we never send polygon data between clients, because that level of display information can be reconstructed within each client, if each client is given the same compact shared scene description.

4.9 Client/server

Our node server implements a shared blackboard model, consisting of a set of state variables, each of which maintains the same value across all clients.

Under the hood, this is implemented as follows: When a client changes the value of a state object, a message is sent from that client to the server encapsulating the change in value. The server then relays that message to all other clients. The effect of these collective messages is to continually update the values of state variables within all clients.

The current value of each state variable is also periodically written as a JSON file on the server. When a new client joins, these updated values are sent to the new client, thereby bringing that client up to date with the XR session in progress.

To support this feature, the server maintains an ordered array of currently active clients, ordered by when each client joined the session. This array is shared with all clients, and is updated whenever any client joins or drops out.

The oldest client has a special responsibility: Whenever a new client joins, the oldest client is responsible for sending updated state values to all clients. The sequence is illustrated in Figure 3. In this case, at first client 2 is the oldest client (A). When client 5 joins (B), it broadcasts a message via the server to all other clients that it has joined the session (C). Client 2 knows that it is the oldest client, and so in response, it broadcasts a message to all other clients with the up-to-date value of the shared state (D). Sometime later client 2 drops out, and now client 4 is the oldest client (E). When client 2 eventually rejoins (F), it broadcasts a message to all other clients that it has joined the session (G). Client 4, now the oldest client, responds by broadcasting a message to all other clients with the up-to-date value of the shared state (H).

4.10 Synchronizing room position

The pose tracking system of most VR headset devices, including the Meta Quest 3 used by our system, does not provide a global coordinate system fixed to the physical environment. Rather, it uses the position of the headset as its coordinate origin, which resets every time the device awakens.

Fortunately, for safety reasons, the Quest 3 requires that there a known boundary be created in every physical room in which the device is operated. This boundary data is made available to developers.

In order to co-locate every VR user on the server to a single common frame of reference, we use each headset's stored boundary information to anchor that headset's frame of reference. By exposing the relative positions of each boundary point to the headset's native origin point, which is the point where the headset was located when it awoke, the system can establish a global coordinate system by saving that position information to the server, tagged with a unique device ID, and comparing the differences between the saved positions and the current positions every time the headset resets its origin point.

The coordinate system is established as a 4x4 matrix, where the first 3 columns are the normalized direction vectors of X, Y, and Z axes respectively, and the last column is the relative position of the

origin point from the current origin position defined by the headset. The elements from the virtual world, including object rendering and physics calculation, will be based on this 4x4 matrix. The transform rules are illustrated in figure 2.

To establish the desired shared reference space, the user simultaneously squeezes the grips of both controllers at a known location in the room, thereby specifying a coordinate system in which the origin is midway between the two controllers, x is the direction from the left controller to the right controller, and y is up. The system records the relationship between this coordinate system and the boundary. Once this relationship has been established, the boundary data stored in the headset suffices to permanently sync the user's frame of reference to the physical room.

With this mechanism in place, a user needs to perform a reference-setting operation only the first time that he/she uses the headset within a physical room. After all headsets have been synced to the same reference system, they will all remain properly synced to that physical room.

4.11 Support for multiple rooms

The core idea of the system is that headsets in the same physical room all share the same virtual content. In the scenario described by this paper, each physical room is labeled by a unique room ID. The system maps the room to its ID when the headset enters the boundary anchored inside that room, thus being able to perform different actions based on the room ID.

When the user enters a physical room that the system has not yet registered, the system prompts the user to enter a room ID. After this ID has been entered, the system maps that ID to the room boundary and saves that mapping to the server.

Any time the headset enters a physical room with a known ID, the system determines which room the headset has entered by comparing the similarities between the geometry of the current boundary to the geometries of all the boundaries stored in the headset. The system computes the distance between the first point on the boundary to all other points in ascending order. The mean square errors of their distance arrays for each stored boundary are computed and sorted. The room ID associated with the boundary with the lowest MSE is then used as the current room ID. Figure 4 illustrates the comparison method.

From the participant's perspective, the resulting experience is that people who are in the same physical room all share the same interactive extended reality overlay. If one participant in that room switches to a different XR overlay, all other participants in the same room are also switched to that overlay.

4.12 Migrating to a shared public server

The local server serving our dynamic website, as discussed thus far, will only be accessible by XR headsets on the same computer network. To move beyond a purely co-located prototype limited to a single network, we deployed our system on a public-facing server with a static IP address, now hosted at https://[HIDDEN FOR ANONYMOUS SUBMISSION]. With this remote server, we can allow remote testing and access, support for future experiences with non-co-located groups of co-located users (for instance, a video call experience between two groups of people in a mixed reality experience between themselves), and real-world use for computers across different networks.

In addition to a public server, there was a need to make it easy for the contributors of our system to see changes reflected in the remote server, and additionally the accessibility of our server. Altogether, the tasks required to make this possible were: (1) setting up a remote server on a static IP that can serve our dynamic website; (2) enabling continuous development of the client/server system; (3) making our system accessible to everyone regardless of technical background.

Using a Virtual Private Server (VPS). To have a secure, dedicated server for our dynamic website, we set up an inexpensive VPS environment on Microsoft Azure [12] by leveraging Azure's Virtual Machine (VM) and Virtual Network services. In the broader system, the VPS simply runs our program and becomes the server, and any computer connecting to the VPS will become a client running our program (refer to client-server section 4.9). To minimize the workload on the VPS, we have designed our system such that the server's primary role is to efficiently pass on messages to clients with minimal processing. This design approach helps avoid the possibility of high CPU usage on the VPS.

A VPS approach gives us the flexibility to increase the bandwidth of our remote server by upgrading and scaling our Azure VM and Virtual Network. However, our current VPS has limited bandwidth and does not implement any Content Development Networks (CDNs) or load balancing across multiple servers. As a result, it suffers from network latency in locations outside the United States and during periods of unusually high traffic. We leave the improvement of our remote server for the future.

Continuous Development. To enable reliability and consistency across our constantly evolving system and the VPS, we enabled a Continuous Development (CD) pipeline. This was done by writing a YAML file [31] that allows pulling from the repository or pushing to the server, and using an automation tool like Ansible or CD ecosystems like GitHub Actions [1] to run said YAML file. When code updates are reviewed and ready for release, the workflow we created handles build and deployment without manual intervention, thus enabling system reliability and DevOps velocity. After every update, the VPS refreshes the program and again becomes the Server of our website without any noticeable changes from the client's point of view.

Security issues and using a domain name. For our system to be accessible, we need to ensure the security of our remote server and to provide a seamless means for any potential user. We therefore created a TLS/SSL certificate for our website and purchased a human-readable domain name, which we then configured to point to our configured server. We also needed the system to support a HTTPS server instead of a HTTP server, and to use Secure WebSocket instead of regular WebSocket [13] message passing.

Continued usage of a local server. While we have a robust remote server that supports the full extensibility and intent of our work, it may be important to note that the local server is still in continuous use and is important for our development. This is done

for expedience, convenience, and the upholding of production standards. More specifically, as with all deployed software, our testing and development of non-production-ready code is handled in local servers and only pushed to production after it has been properly reviewed and tested.

5 USE CASE

In this section, we explore a variety of potential use cases that demonstrate the versatility and capability of our co-located XR system, illustrating how it can be applied in diverse scenarios and environments.

5.1 Adding shared virtual objects to the physical world

Statue: We experimented with users observing various realistic objects that have been scanned from the physical world, and that appear to be in physical contact with real objects. In the accompanying video, two participants each see what appears to be a small realistic statue sitting on a pedestal. The pedestal is real, but the statue is virtual. The video is being captured by a third participant who is wearing a Meta Quest 3 headset (see Fig.1a).

Hypercube: A human view of a four-dimensional object is necessarily a projection down to our own three dimensions. On a traditional computer display, there is a further flattening down to the two dimensions of the screen. By representing a hypercube as a true three-dimensional object, we avoid this second loss of dimensionality. In our implementation, each participant can use linear movements in x, y, and z of their hand or controller to rotate the hypercube about, respectively, its yz, zx, and xy hyper-axes.

Chart: This is an example of how info-graphics might appear in a shared extended reality view. Animated charts and similar data displays can be fully 3D, which allows a richer and more immersive display of information than is practical on a 2D screen. The accompanying video demonstrates several options for text display. For the labeled axes the text rotates to always face the participant. The display of time is an example of an alternate format in which the text appears on the four sides of a transparent display box.

Window: Multi-participant XR allows us to share objects that appear real yet turn out to have magical properties. In this case, all participants see a window in the middle of the room. When viewed from one side, it appears to be an ordinary window frame. Yet when viewed from the other side, it shows a view into another world. To implement this, we use the clipping mechanism that is described elsewhere in this paper.

Moon: Another example of magical realism is the ability to place a realistic 10 foot diameter moon above the heads of all participants. Although it starts up in the air, participants can pull the moon down, thereby bringing it into the shared physical space.

Skylight: Another use of clipping is a virtual skylight. In this case, all participants see an overhead sky within a rectangular frame on the ceiling. The sky seems to be infinitely far away (see Fig.1b).

Crayon: The capabilities of Chalktalk [18] have been replicated in multi-participant XR. A custom fragment shader uses procedural noise to create the effect of a transparent purple crayon inspired by Harold and the Purple Crayon [8]. After drawing a shape, a participant can then click on that shape and the shape will come to life and become an animated object.

5.2 Using physical objects as interaction devices

Music: One useful feature of extended reality is that physical objects can be used as interaction devices, which allows for realistic haptic feedback. In this case, the midi output from a portable music keyboard is used to generate colored objects that appear to rise up from the keyboard in response to notes being played. All participants see these colorful objects in the same location, creating the sense that the objects are being generated by the keyboard itself (see Fig.1c).

Code editing: In our system, any participant can edit an animated scene by bringing up a code window and doing live coding on the underlying Javascript that defines the scene. This can be done, as shown in the accompanying video, by typing on a physical Bluetooth keyboard that is connected to a WebXR client. In practice, this allows us to place Bluetooth keyboards in various convenient places, so that the code which defines scenes can be quickly edited while wearing an XR headset, without the need to sit in front of a computer (see Fig.1d).

In the accompanying video, all participants can see the editable code window as a transparent XR object that each participant perceives as always turned to face toward them. Progressive changes to the code made by the participant who is at the keyboard cause objects in the shared virtual scene to immediately change within the views of all participants.

5.3 Symmetrical collaboration

Puzzle: Two participants cooperatively solve a 3D puzzle together, a 3x3x3 variant of the classic 4x4 **15 puzzle**. There is a shared state stored in the server as a JSON string that is synced across all clients, enabling all players to experience the game together in real time. To illustrate the rules of the game, an info box with text instructions floats above the puzzle object. The contents of the associated info box also change when the players win the game.

Construction: This is an example of a multi-participant construction toy. Each participant can create a ball by clicking with their controller (or pinching with one hand) in the empty air. Once created, a ball is moved by dragging it, and deleted by clicking on it with a controller or pinching it with a hand (see Fig.1f).

Clients update the shared state by sending messages indicating either that a ball should be placed at a particular location, or that a ball should be deleted. Each message contains fields id, op, pos indicating, respectively, the id of the ball, the operation to be performed, and the ball's target position. At each render frame, the following code is run on each client to synchronize their own copy of the scene Model to the shared scene Model:

```
server.sync('state_object', messages => {
   for (let id in messages) {
      let message = messages[id];
      if (message.op == 'delete')
```

```
            delete state_object[message.id];
        else
            state_object[message.id] = message.pos;
    }
});
```

This example illustrates an important principle of our client/server architecture: Even though the object array that constitutes the shared state can itself be quite large, the messages passed between clients to continually update that shared state are each small, granular, and lightweight.

Chess: Multi-participant chess was designed to allow either a standard tabletop-size chess game or else a giant chess game in which players can physically walk around on the chess board, picking up giant pieces to move them. In the latter case, the chess pieces are made transparent. Even though participants are technically being rendered behind the chess pieces in the video passthrough layer, in practice participants do not notice this, because the transparency creates the perceptual illusion that other participants are simply walking through a shared transparent world (see Fig.1h).

5.4 Asymmetrical collaboration

Our XR classroom explores two different paradigms of co-located XR teaching/learning experiences.

Classroom: In the symmetric version, both the instructor and the students are oriented towards a shared board. This board displays the instructional material and quiz content for the entire class. This arrangement allows for a unified view of the content for both the teacher and the students.

In contrast, the asymmetric version creates a different experience for participants. Although everyone is looking at the same board, the displayed content varies depending on the viewer. Students see quiz questions and answer options, which they can interact with. Simultaneously, the teacher has access to a different view that shows the collective performance of the class. This setup enables the teacher to provide personalized tutoring, tailoring assistance to individual students based on their specific performance in the quiz.

Balance Training. In this two-person XR experience tailored for physical therapy, therapists utilize controllers to place target elements, red balls, into the virtual environment. The objective is to facilitate balance and stability exercises for patients. Therapists can strategically position the red ball within the XR space, challenging patients to reach for it. The target elements can be placed in any arbitrary position without worrying about the physical setup constraints. As the therapy progresses, therapists gradually extend the distance of the red ball, encouraging patients to step out of their base support and enhance their balance capabilities. This interactive approach adds an engaging and immersive experience to traditional physical therapy and allows therapists to tailor exercises to individual patient needs, enhancing a personalized and effective rehabilitation experience within the XR co-located setting (see Fig.1e).

5.5 Demonstrations of rendering capability

Snow: This scene consists of 50,000 texture-mapped particles, with procedural movement applied to each particle. Because the movement of each particle depends only on its physical location and clock time, all clients can see the same arrangement of particles, without the need to explicitly transmit individual particles between clients (see Fig.1g).

Smoke: In this example, all participants see a volumetric procedural texture of a curl of smoke rising from a virtual cigarette. In practice, each participant is actually viewing three transparent planar slices, located at different distances, of the volumetric texture. The slices always rotate to face that participant. This creates the subjective effect for each participant that they are looking at a real–time volumetric time-varying texture from their own unique point of view.

6 DISCUSSION AND CONCLUSION

In this paper, we propose a system focused on providing an open-source, powerful, and easy-to-use XR platform. Specifically, the platform enables and supports co-located multi-person experiences, and we introduced the design needs of such a system, and the implementation details in realizing the design goals. We discussed, in-depth, the architecture that enabled co-location including our client-server architecture and the methodology to enable room synchronization between multiple users, and demonstrated various capabilities of the platform via XR experiences that we created using the platform.

Our current method has several limitations. It is not a rendering engine that was created for or has support for, heavy computer graphics software, such as triple-A video games. Rather, it is powerful enough to make standalone XR scenes and is suitable for use in research on co-located user experiences in XR. Additionally, there is no depth data or camera data available provided by mainstream XR headsets for our WebXR platform, which limits the capabilities of our platform.

As for future work, we plan to make our platform even more powerful by adding our own hardware in addition to Meta Quest headsets. One example is having our system communicate with devices such as drones for active tangibles. Another example is cameras that allow for access to video feed and depth data. This would open possibilities for experiences with scene and world understanding, and enable techniques like occlusion or dis-occlusion.

Also, we plan to incorporate spatial audio and TTS (Text-to-Speech) recognition, along with an out-of-the-box Transformer-based AI layer for collaborative content creation. For example, if collaborators are designing a house, their acts of creation and design will simply be woven into the conversation. One collaborator might say "The roof should be pointed and green," and another might add "The driveway curves this way, and a dog is running across the lawn," and these things will immediately appear. To support such interactions in XR, it is important to have a robust and flexible substrate.

REFERENCES

[1] C. Chandrasekara and P. Herath. 2021. *Hands-on GitHub Actions: Implement CI/CD with GitHub Action Workflows for Your Applications.* Apress.

[2] Esteban WG Clua, Daniela G Trevisan, Thiago Porcino, Bruno AD Marques, Eder Oliveira, Lucas D Barbosa, Thallys Lisboa, Victor Ferrari, and Victor Peres. 2020. Challenges for XR in Games. In *Forum on Grand Research Challenges in Games and Entertainment*. Springer, 159–186.

[3] Åsa Fast-Berglund, Liang Gong, and Dan Li. 2018. Testing and validating Extended Reality (xR) technologies in manufacturing. *Procedia Manufacturing* 25 (2018), 31–38.

[4] Moinak Ghoshal, Juan Ong, Hearan Won, Dimitrios Koutsonikolas, and Caglar Yildirim. 2022. Co-located Immersive Gaming: A Comparison between Augmented and Virtual Reality. In *2022 IEEE Conference on Games (CoG)*. IEEE, 594–597.

[5] Immersive Web Working Group. 2022. WebXR Device API. W3C Working Draft. https://www.w3.org/TR/webxr/.

[6] Botao Hu, Yuchen Zhang, Sizheng Hao, and Yilan Tao. 2023. InstantCopresence: A Spatial Anchor Sharing Methodology for Co-located Multiplayer Handheld and Headworn AR. In *2023 IEEE International Symposium on Mixed and Augmented Reality Adjunct (ISMAR-Adjunct)*. IEEE, 762–763.

[7] Botao Hu, Yuchen Zhang, Sizheng Hao, and Yilan Tao. 2023. MOFA: Exploring Asymmetric Mixed Reality Design Strategy for Co-located Multiplayer Between Handheld and Head-mounted Augmented Reality. In *Extended Abstracts of the 2023 CHI Conference on Human Factors in Computing Systems*. 1–4.

[8] Crockett Johnson. 1992. *Harold and the Purple Crayon*. HarperCollins Children's Books.

[9] Steve Mann, Tom Furness, Yu Yuan, Jay Iorio, and Zixin Wang. 2018. All reality: Virtual, augmented, mixed (x), mediated (x, y), and multimediated reality. *arXiv preprint arXiv:1804.08386* (2018).

[10] Mark McGill, Jan Gugenheimer, and Euan Freeman. 2020. A quest for co-located mixed reality: Aligning and assessing slam tracking for same-space multi-user experiences. In *Proceedings of the 26th ACM Symposium on Virtual Reality Software and Technology*. 1–10.

[11] Meta Platforms, Inc. 2022. *Meta Quest Documentation*. Meta Platforms, Inc. https://developer.oculus.com/documentation.

[12] Microsoft. 2015. *Microsoft Azure*.

[13] G. L. Muller. 2014. HTML5 WebSocket protocol and its application to distributed computing. *ArXiv* 1409.3367 (Sep. 2014).

[14] Susanna Nilsson, Björn JE Johansson, and Arne Jönsson. 2009. A co-located collaborative augmented reality application. In *Proceedings of the 8th International Conference on Virtual Reality Continuum and Its Applications in Industry*. 179–184.

[15] Frédéric Noel and Daniel Brissaud. 2003. Dynamic data sharing in a collaborative design environment. *International Journal of Computer Integrated Manufacturing* 16, 7-8 (2003), 546–556.

[16] Kaitlyn M Ouverson and Stephen B Gilbert. 2021. A composite framework of co-located asymmetric virtual reality. *Proceedings of the ACM on Human-Computer Interaction* 5, CSCW1 (2021), 1–20.

[17] Vasco Pereira, Teresa Matos, Rui Rodrigues, Rui Nóbrega, and João Jacob. 2019. Extended reality framework for remote collaborative interactions in virtual environments. In *2019 International Conference on Graphics and Interaction (ICGI)*. IEEE, 17–24.

[18] Ken Perlin. 2016. Future Reality: How Emerging Technologies Will Change Language Itself. *IEEE Computer Graphics and Applications* 36, 3 (May 2016), 84–89. https://doi.org/10.1109/MCG.2016.56

[19] Henrik Lundqvist Qvarfordt, Christer and Georgios P. Koudouridis. 2018. High quality mobile XR: Requirements and feasibility. *2018 IEEE 23rd International Workshop on Computer Aided Modeling and Design of Communication Links and Networks (CAMAD)* 5 (2018), 1–6.

[20] Jiten Rama and Judith Bishop. 2006. A survey and comparison of CSCW groupware applications. In *Proceedings of the 2006 annual research conference of the South African institute of computer scientists and information technologists on IT research in developing countries*. 198–205.

[21] Xukan Ran, Carter Slocum, Yi-Zhen Tsai, Kittipat Apicharttrisorn, Maria Gorlatova, and Jiasi Chen. 2020. Multi-user augmented reality with communication efficient and spatially consistent virtual objects. In *Proceedings of the 16th International Conference on emerging Networking EXperiments and Technologies*. 386–398.

[22] Daniel Roth, Constantin Klelnbeck, Tobias Feigl, Christopher Mutschler, and Marc Erich Latoschik. 2018. Beyond replication: Augmenting social behaviors in multi-user virtual realities. In *2018 IEEE Conference on Virtual Reality and 3D User Interfaces (VR)*. IEEE, 215–222.

[23] Miriam Sturdee, Hayat Kara, and Jason Alexander. 2023. Exploring Co-located Interactions with a Shape-Changing Bar Chart. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. 1–13.

[24] Miguel A Teruel, Elena Navarro, Pascual González, Víctor López-Jaquero, and Francisco Montero. 2016. Applying thematic analysis to define an awareness interpretation for collaborative computer games. *Information and Software Technology* 74 (2016), 17–44.

[25] Anna Vasilchenko, Jie Li, Bektur Ryskeldiev, Sayan Sarcar, Yoichi Ochiai, Kai Kunze, and Iulian Radu. 2020. Collaborative learning & co-creation in XR. In *Extended Abstracts of the 2020 CHI Conference on Human Factors in Computing Systems*. 1–4.

[26] Keru Wang, Zhu Wang, and Ken Perlin. 2023. Asymmetrical VR for Education. In *ACM SIGGRAPH 2023 Immersive Pavilion*. 1–2.

[27] Keru Wang, Zhu Wang, Karl Rosenberg, Zhenyi He, Dong Woo Yoo, Un Joo Christopher, and Ken Perlin. 2022. Mixed Reality Collaboration for Complementary Working Styles. In *ACM SIGGRAPH 2022 Immersive Pavilion*. 1–2.

[28] Lihui Wang, Andrew Yeh Chris Nee, Andrew Yeh Chris Nee, and Lihui Wang. 2009. *Collaborative design and planning for digital manufacturing*. Springer.

[29] Viktor Wendel, Michael Gutjahr, Stefan Göbel, and Ralf Steinmetz. 2013. Designing collaborative multiplayer serious games: Escape from Wilson Island—A multiplayer 3D serious game for collaborative learning in teams. *Education and Information Technologies* 18 (2013), 287–308.

[30] Xuanhui Xu, David Kilroy, Antonella Puggioni, and Abraham G Campbell. 2022. Co-located mixed reality for teaching equine radiology techniques to veterinary students. In *2022 IEEE Games, Entertainment, Media Conference (GEM)*. IEEE, 1–6.

[31] YAML. n.d.. *YAML Ain't Markup Language (YAMLTM) Version 1.1*. https://yaml.org/spec/1.1/

[32] José P Zagal, Jochen Rick, and Idris Hsi. 2006. Collaborative games: Lessons learned from board games. *Simulation & gaming* 37, 1 (2006), 24–40.

A APPENDICES

Fig. 1. **Use case demonstrations and capability of our multi-participant co-located WebXR system:** a. Users viewing shared virtual statue anchored in the physical environment; b. A shared virtual window; c, d. Physical objects such as musical keyboards and computer keyboards are used as interaction input devices; e. The system supports asymmetrical interaction, such as a physical therapist can set up balance tasks for their patient; f. The system can synchronize render complicated scenes such as a snow scene with 50,000 textured particles; g, h. Users can do co-located collaborative interactions, such as constructing things and playing virtual chess.

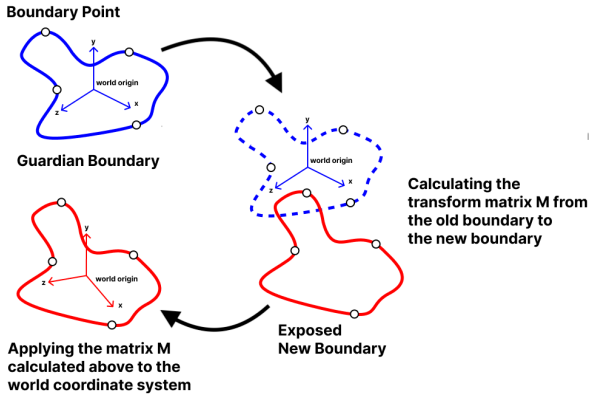

Fig. 2. The world coordinate is anchored with the boundary

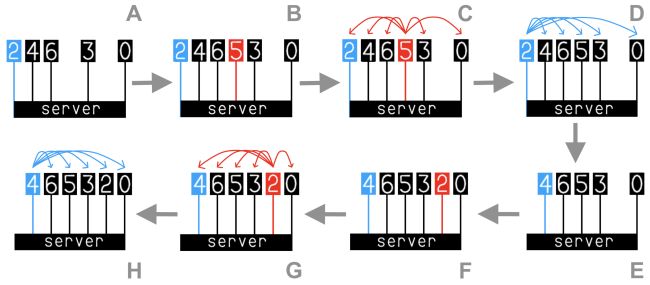

Fig. 3. On-boarding new clients

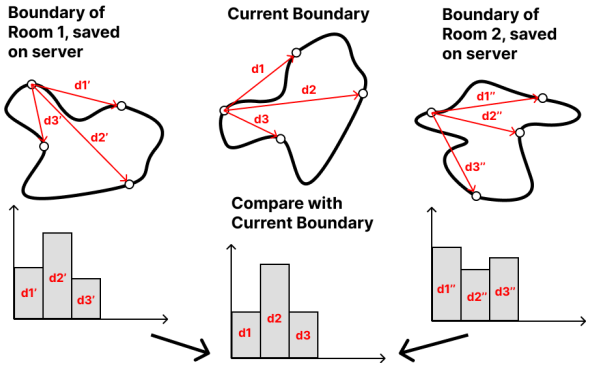

Fig. 4. Identifying the room from its boundary shape